Con(PA) Turing machines: comparison, verified intermediate constructions, and the remaining state-reduction obstruction

Research and reproducibility report

13 November 2026

Abstract

The requested machine with fewer than 196 working states was not obtained. This report does not certify a 195-state machine. It compares the supplied 208-, 209-, and 196-state designs at pinned revisions, accounts for the different 208-state presentations, and records the new work in a form that can be checked and continued. The principal complete intermediate combines a cached-head-only Shift test with an MP result stored in that cache. It has 196 binary working states, 3,788 source instruction occurrences, an exact 57,922-step blank startup, and 4,928 uniform source-instruction contracts. A written proof supplies its PA-provable consistency calibration; it is distinct from a new Lean kernel-checked endpoint. A repaired 157-node source layout preserves the old 3,812-instruction control graph, but the verified physical allocation has 207 states. A direct-negation specialization reduces the source to 3,556 instructions and has a verified 197-state realization. An even-tag formula recoding justifies a simpler parity acceptance condition, but no sub-196-state implementation or complete parity startup is certified here.

Contents

1 What was requested, and what is established **3**

2 Pinned inputs and comparison method **3**
2.1 Revisions and archive coverage . . . 3
2.2 Canonicalizing machines without confusing syntax and behavior . . . 4

3 How the supplied designs differ **4**
3.1 The 208-state family has more than two classes . . . 4
3.2 The two 209-state ancestors must be distinguished . . . 5
3.3 PA196 is a logical and compilation change . . . 5

4 The logical strength and the meaning of "provable in PA" **6**

5 The new complete intermediate: Combined196 **7**
5.1 The six physical changes . . . 7
5.2 Exact startup and uniform runtime certificate . . . 7

6 Uniform arithmetic contracts for the combined source **8**
6.1 Exact source changes and proof reuse . . . 8
6.2 Shift: clearing the old top costs $2t + 1$ instructions . . . 8
6.3 MP: the success branch is a single transfer loop . . . 9
6.4 From these contracts to the literal binary table . . . 9

7 A complete command compiler for the combined Shift and MP changes 10
7.1 The changed arithmetic commands . . . 10
7.2 Stack words and the changed pop contract . . . 10
7.3 All axiom selectors remain total . . . 11
7.4 The new modus ponens suffix . . . 12
7.5 A primitive recursive compiler and universal halting word . . . 12
7.6 Soundness and PA-provable calibration . . . 13

8 Direct negation: a shorter source with a 197-state realization 13

9 Why the repaired 157-node source still costs 207 states 14

10 An even-tag recoding that permits parity acceptance 15
10.1 The new formula codes . . . 15
10.2 Why every odd acceptance yields a PA refutation . . . 15
10.3 Transporting the old PA-refutation compiler . . . 16
10.4 Command compilation with the cached-head parity test . . . 16
10.5 A residue-one acceptance test without recoding . . . 17

11 Physical experiments for the parity architecture 17

12 ConciseIPRA: useful ideas and the strength barrier 18

13 Rejected shortcuts and exact search limits 18

14 Reproduction and certificate boundaries 19

A The reverse consistency implication 21
A.1 An arithmetical interpretation of the exact calculus . . . 21
A.2 Uniform finite-comprehension proofs . . . 21
A.3 Translation of the gate and the logical heads . . . 22
A.4 From a finite halting trace to a refutation . . . 23

1 What was requested, and what is established

The goal is a deterministic binary Turing machine M on a single two-way infinite tape, started on the all-zero tape, with fewer than 196 working states and

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M) \longleftrightarrow \mathrm{Con}(\mathrm{PA}).$$

The distinguished halting label is not a working state. Every working state has exactly two transition rows, one for each scanned symbol; a transition writes a bit and moves exactly one cell left or right. Changing the alphabet, using a preloaded tape, or counting source DAG nodes as machine states would not meet this goal.

The requested strict inequality remains open in this investigation. The accepted complete intermediate in this packet is denoted $M_{196}^{\mathrm{combined}}$. Its transition table is `machines/h_only_mp_h.tm`. The file contains every transition, not a patch requiring an unspecified ancestor. Its control labels are $0, \ldots, 195$, its start label is 0, and its terminal label is 196.

Object	Working states	Source sites	Status in this packet
Pinned PA196	196	3812	Reference table and uniform checks
Combined Shift/MP	196	3788	Complete written proof and checks
Direct negation variant	197	3556	Equivalent arithmetic macro
Repaired 157-node source	207	3812	Same source CFG; verified realization
Parity control, reset length 8	198	3801	Runtime component only
Requested construction	≤ 195	—	Not obtained

There are three different kinds of evidence throughout this report. Exact finite checks establish table counts, hashes, local rewrites, finite source bindings, and the literal startup configuration. Uniform source-site checks use symbolic tape runs and the unchanged register-engine lemmas to cover arbitrary natural register values. Written arithmetic arguments establish the changed commands and the proof compiler. None of these is described as a completed new Lean formalization of the modified machine.

2 Pinned inputs and comparison method

2.1 Revisions and archive coverage

All comparisons use the following revisions, rather than a moving main branch:

Repository/version	Commit
BusyBeaver, PA196	`407cdc356f6b586635e71ac92be394255f4bfea4`
BusyBeaver, PA209	`69d98241a721ead20b4f0b17df9062950b40fbfe`
ProveIt, incoming and ConciseIPRA	`dc523e7b06ffb414093c90aa035500ca283e5f75`

The current incoming directory contains 18 top-level ZIP archives. The inventory in `provenance/current_archive_inventory.json` records their exact hashes, byte sizes, entry counts, and report paths. The GitHub blob identities were checked when materializing these archives. For every selected 208-state comparison table, the packet additionally records every matching current ZIP member in `provenance/comparison_input_verification.json`. This prevents an old scratch copy from silently substituting for the current supplied input.

The seven newer reports in the incoming directory contain prior reductions, negative experiments, and repeated copies of earlier tables. They are part of the literature being compared. Their already-recorded unsuccessful 195-state candidates, HA-strength modification of ConciseIPRA, and written internal-converse arguments are not claimed as discoveries of this report. The full source archives total well over 100 MiB; the deliverable retains selected exact tables, relevant source, hashes, provenance, and notices instead of recursively duplicating those archives.

2.2 Canonicalizing machines without confusing syntax and behavior

The canonical comparison traverses the rooted transition graph in breadth first order, taking the read-0 edge before the read-1 edge. It records, for each edge, the written bit, move direction, and canonical successor label, with a distinguished terminal marker. Equality of these encodings is rooted, action-preserving graph isomorphism. It is stronger than equal state count and weaker than deciding whether two different machines have the same halting behavior. It is a finite structural test, not a solution to machine equivalence.

`comparison/compare_designs.py` checks input hashes and complete tables, performs this canonicalization, and checks the extra local helper blocks. It runs both normally and under `python3 -O`; its essential comparisons use explicit exceptions rather than removable Python assertions.

3 How the supplied designs differ

3.1 The 208-state family has more than two classes

Nine designated presentations in the five original U208 packets collapse to two rooted graph classes, called A and B in the comparison receipt. However, `pa208_research_2026-10-29.zip` supplies 18 additional helper layouts: nine based on A and nine based on B. Each has a distinct rooted transition graph, and none is isomorphic to A or B. The correct designated comparison therefore has

$$9 + 18 = 27 \text{ presentations}, \qquad 2 + 18 = 20 \text{ graph classes}.$$

Saying that all supplied 208-state machines form only two classes would omit those 18 layouts. Copies embedded in later research packets do not create further designated designs merely by being copied.

Original packet (all dated 2026-10-29)	Primary class	Alternative class
`U208_ConPA`	A	—
`U208_ConPA_Conditional_Context`	A	B
`U208_ConPA_Entry_Context`	A	B
`U208_ConPA_History`	A	B
`u208_research`	B	A (plain variant)

The common original 208-state reduction identifies the historical native controls 172 and 211. Their entry histories expose different predecessor bits, so the merged control can recover the required choice from the tape. One presentation keeps 172 and removes 211; another naming keeps 211. These are presentations of the same saving, not independent savings that can be stacked to obtain 207 states. The four whole-tape local blocks have lengths 7, 1, 3, and 1 in the representative audit. With the historical state labels, the stuttering-time relation is

$$T_{208} = T_{209} + 6N(172, 0) + 2N(211, 0), \qquad T_{209} \le T_{208} \le 7T_{209}.$$

Here $N(q, b)$ counts the old trace's visits to the indicated read case. The relation concerns the designated U209 parent and its U208 descendant. It must not be applied by label name to James's different PA209 table.

The extra 18 helper layouts identify another pair and add a helper control, so their net state count remains 208. Their finite helper blocks are replayed in the comparison script. They give useful different sharing arrangements, but no additional state saving by themselves.

Layout	Drop	Keep	Read	New helper	Helper on 0	Helper on 1
A 1	301	88	0	500	$0R122$	$1R500$
A 2	127	124	0	500	$0R8$	$1R59$
A 3	127	124	0	500	$0R8$	$1R109$
A 4	127	124	0	500	$0R8$	$1R125$
A 5	190	160	1	500	$0R86$	$1R193$
A 6	178	168	0	500	$0R154$	$1R176$
A 7	178	168	0	500	$0R154$	$1R225$
A 8	178	168	0	500	$0R159$	$1R176$
A 9	178	168	0	500	$0R159$	$1R225$
B 1	301	88	0	500	$0R122$	$1R500$
B 2	127	124	0	500	$0R8$	$1R59$
B 3	127	124	0	500	$0R8$	$1R109$
B 4	127	124	0	500	$0R8$	$1R125$
B 5	190	160	1	500	$0R86$	$1R193$
B 6	178	168	0	500	$0R154$	$1R176$
B 7	178	168	0	500	$0R154$	$1R225$
B 8	178	168	0	500	$0R159$	$1R176$
B 9	178	168	0	500	$0R159$	$1R225$

The helper table uses the native labels of its designated A or B baseline. The kept control reads the predecessor bit through the new helper on the listed differing branch; the comparison script checks the complete patch and all four old-state/read-bit cases, including simultaneous target redirection.

3.2 The two 209-state ancestors must be distinguished

The supplied U208 packets descend from their own U209 parents. Those parents are not literally James's PA209 machine at the historical commit. They share an earlier architecture, but source bracketings, selected sharing, labels, and startup traces differ. A direct subtraction of numerical labels across the two branches is consequently invalid.

Design	States	Rows	Source sites	Reported/checked startup
Original U208 A/B	208	416	3909	48140
James PA209	209	418	3948	58210
James PA196	196	392	3812	58154
Combined196	196	392	3788	57922

The U208 source has 5,083 resolved graph edges and 5,084 outcome contracts in the inherited reports. PA196 has 4,959 outcome contracts, counting both zero and positive cases of each decrement. The startup counts are exact reaches of specific clean frames, not useful-run benchmarks or evidence of nonhalting. The PA196 and Combined196 startups are replayed in this packet; the older counts are attributed to their pinned construction certificates and comparative sources.

3.3 PA196 is a logical and compilation change

PA196 is not obtained by simply applying all the old U208 identifications to PA209. In native numbering, its comparison with James's PA209 removes 75 labels, introduces 62 new 1300-series labels, and keeps 134 labels. Among those retained labels, 265 transition rows are identical. The only three retained-row changes are

native row	PA209	PA196
$(101, 1)$	$1R115$	$1R1367$
$(164, 0)$	$0L311$	$0R77$
$(190, 1)$	$1L308$	$1R99$.

Both old controls 172 and 211 are already absent. Reapplying their old identification cannot reduce the 196-state table.

The physical allocation consists of 36 engine states, 13 leaf-routing states, and 147 reader states representing 159 distinct source nodes. The AX body is a 201-item macro word. Its 68 distinct new reader nodes are represented by 62 physical controls through six contextual folds. The overall source contains further sharing outside AX.

Its constructors use Cantor pairing

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(a,b) = 4p(a,b), \quad M(a,b) = 4p(a,b)+1,$$

$$U(a,B) = 4p(B,a)+2, \qquad E(a,b) = 4p(a,b)+3.$$

The transposition in U is intentional. Code 0 acts as truth and code 1 is the chosen contradiction. The logical changes include moving equality vacuity into the guarded separation bundle, removing the old separate ρ_7 role, and changing some head orientations and their ordering. These changes shorten the AX word and improve physical sharing. The exact definitions, rather than a verbal summary, are the specification; see `PA196/Commands/Code.lean` in the pinned source [1].

4 The logical strength and the meaning of "provable in PA"

Let $\mathcal{C}$ be the numerical calculus searched by PA196. Four separate claims are needed for the requested calibration:

1. PA refutations translate to $\mathcal{C}$ refutations.
2. $\mathcal{C}$ refutations translate syntactically to PA refutations.
3. A $\mathcal{C}$ refutation yields a halting command word that is usable by the exhaustive scheduler, whatever store previous trials have left.
4. The literal blank-tape machine and the source program simulate one another for halting, with an exact startup and positive finite steps.

For an internal biconditional, these transformations and their correctness must be available within PA, not merely as true external assertions.

The upstream file `PA196/Independence.lean` includes an internal theorem named `pa_proves_nonhalting_consistency`. Its statement is

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \longrightarrow \mathrm{Con(PA)}.$$

It also exports nonprovability and two-sided independence, using the separate external soundness/model development. This should not be described as an exported Lean proof of the displayed biconditional. In particular, external soundness of a calculus does not by itself supply the reverse implication inside PA.

The reverse internal direction has a written syntactic argument in the earlier incoming research, reproduced with exact provenance in Appendix A. Decode formulas by a primitive recursive arithmetization, interpret membership as binary-bit membership of natural numbers, and compile each calculus axiom and inference into a PA proof of its arithmetic translation. Finite-comprehension instances are justified by arithmetic induction on the finite bound. This constructs proofs of individual translated instances; it does not install a truth predicate for arbitrary arithmetic formulas. A numeric-calculus proof of code 1 then yields a PA proof of a fixed contradiction. Correctness is established by induction on the coded derivation.

The full calibration is consequently a written mathematical theorem with explicit finite and uniform components, while the absent endpoint remains a formalization task. The new combined construction below retains the same calculus and adapts its command compiler. It does not claim

that a new Lean endpoint has been built or that the upstream project was rebuilt in this run. Selected upstream definitions and proof statements are included for review; complete dependency reproduction follows the pinned repository's own instructions.

5 The new complete intermediate: Combined196

5.1 The six physical changes

Starting with the dense PA196 table, change precisely these six rows:

State	Read	Write	Move	Next
115	1	1	R	108
37	0	0	R	11
37	1	1	R	101
127	0	0	R	133
127	1	1	R	20
133	0	0	R	120

The first change moves the MP success result into the cached head. The remaining changes implement the Shift variant that tests the cached head alone. The existing single halting row, $(63, 1) \mapsto (1, R, 196)$, remains. The full table's SHA-256 is

`bd74754d75e7b8ba97fa7bfc104374de33ae603fc183ae7f08f053ab8c9ea592`.

The two component changes were already present separately in prior research; the new result here is their simultaneous realization and the completed command compiler for the resulting behavior. Combining the patches without updating that compiler would leave a logical gap. In particular, Pop becomes partial at a saved top of 1, and the MP compiler needs a dummy zero beneath its premises to preserve the underlying stack.

The combined source has 159 distinct internal DAG nodes and 3,788 leaf occurrences, compared with 159 and 3,812 in the original. Thus 24 fewer source occurrences do not imply any fewer physical states. The longest source address is 39 bits. All 196 working controls are graph reachable.

5.2 Exact startup and uniform runtime certificate

The table reaches root control 75 with head at absolute coordinate -2 after exactly 57,922 literal transitions. The whole tape is checked against the clean frame at source site 25. Relative to the head, that frame consists of a left marker at -1, the path `1000000`, enough zeros to put the file marker at 48, and the encoded unary register file. The physical values are

$$[0, 0, 0, 0, 0, 0, 2, 0, 0] +\!\!+ [1]^{26}.$$

Under the physical register permutation $[6, 2, 0, 4, 3, 7, 8, 5, 1]$, the logical candidate register is 2 and all other live logical registers are zero. The test checks the complete finite support of the tape, not just the first several cells or the fact that control 75 was visited.

The symbolic verifier checks 4,928 instruction outcomes over every natural register file and every additional clean gap $n \geq 0$, with total gap at least 48. A symbolic zero run has length $n + c$; the remaining file is an opaque arbitrary register-file object. Scan lemmas and four register-effect summaries stand for unbounded loops. Their hypotheses are checked against the unchanged engine rows before use. The maximum symbolic trace length is 524 verifier steps. This number is a proof-checking statistic, not a bound on literal execution for large register values.

The union of the literal startup rows and the rows visited in symbolic source execution covers every one of the 392 transition rows. No state or row can be dropped by the simple unused-row argument under this certificate. The arithmetic source-macro proof and command-level calibration are given in Sections 6 and 7.

6 Uniform arithmetic contracts for the combined source

The following closes the source-macro part of the argument for the combined 196-state intermediate machine. It uses induction on arbitrary natural register values, rather than bounded executions of the source program. The accompanying `check_source_binding.py` verifies the finite resolved-instruction identities to which those inductions apply. No new Lean theorem is claimed.

6.1 Exact source changes and proof reuse

The combined tree has 3788 leaves. Its resolved instructions at ordinals 0 through 3772 agree with the 3812-leaf upstream source except for three relocated outgoing edges:

$$48: \ 3800 \mapsto 3777, \qquad 3692: \ 3799 \mapsto 3776, \qquad 3772: \ 3799 \mapsto 3776.$$

At ordinal 48 only the zero branch of the decrement changes. Consequently the root guard (0–10), stack decoder (11–24), candidate fetch (25–47), AX body (50–3692), and MP prelude (3693–3772) retain their arithmetic effects. The proof of each reused fragment transports by its unchanged instruction rows, with the indicated exit renamed. The two entry points that change their arithmetic behavior are the MP success continuation and Shift.

The logical registers are $T = R_1$, $W = R_2$, $D = R_4$, $H = R_5$; $R_6 = X$ and $R_7 = Y$ are scratch. The source command interface has $R_3 = X = Y = 0$. The candidate number R_0 and unconsumed command word R_8 are framed through the command operations except when explicitly read by the unchanged fetch or operand-decoding routines.

6.2 Shift: clearing the old top costs $2t + 1$ instructions

The new Shift entry is 3777. Its resolved tail, using dec with zero destination first, is

3777	$\mathrm{dec}(T, 3779, 3778)$
3778	$\mathrm{jump}(3777)$
3779	$\mathrm{dec}(H, 3783, 3780)$
3780	$\mathrm{inc}(T, 3781)$
3781	$\mathrm{inc}(X, 3782)$
3782	$\mathrm{jump}(3779)$
3783	$\mathrm{dec}(X, 3786, 3784)$
3784	$\mathrm{inc}(H, 3785)$
3785	$\mathrm{jump}(3783)$
3786	$\mathrm{inc}(X, 3787)$
3787	$\mathrm{jump}(0)$.

Starting with $(T, H, X) = (t, h, 0)$, the first loop reaches 3779 with $(0, h, 0)$ in exactly $2t + 1$ instructions. Its induction step decreases T once and executes its one return jump. The second loop reaches 3783 with $(h, 0, h)$ in $4h + 1$ instructions; each iteration decreases H and increases T and X. The third reaches 3786 with $(h, h, 0)$ in $3h + 1$ instructions. All other registers are unchanged throughout. Thus the exact pre-guard duration is

$$(2t + 1) + (4h + 1) + (3h + 1) = 2t + 7h + 3.$$

The last two rows enter the unchanged root guard with $X = 1$. That guard compares H with one, consumes H and X, and either halts or enters the unchanged stack decoder at 11. The existing guard proof applies with its initial value h, instead of the old value $t + h$. Including rows 3786 and 3787, its duration is eight instructions when $h = 1$, and $2h + 7$ instructions otherwise. Hence:

$h = 1$: Shift halts in $2t + 7h + 11$ source instructions;
$h \neq 1$: Shift reaches decoder 11 in $2t + 9h + 10$ source instructions, $(T, H, X) = (h, 0, 0)$.

The decoder preserves T, D, changes W to its Cantor tail, and puts its Cantor head in H. It has a finite positive execution for every natural W. This proves the complete modified Shift contract, including arbitrary t, w, d, h and preservation of the other registers.

6.3 MP: the success branch is a single transfer loop

Let the incoming offset field be x, and set $y = d + x$. The unchanged operand decoder and MP prelude consume that field and reach one of two exits, with $(T, W, D, H, X, Y) = (0, w, y, 0, 0, 0)$. Equality $I(t, y) = h$ selects 3773; inequality selects 3776. The new tail is

3773	dec$(D, 3776, 3774)$
3774	inc$(H, 3775)$
3775	jump(3773)
3776	jump(25).

Induction on y gives the success-loop invariant $(D, H) = (y - k, k)$ after k iterations. There are three instructions per iteration and one final zero decrement, so the loop reaches 3776 in $3y + 1$ instructions with $(T, W, D, H) = (0, w, 0, y)$. Its return to fetch costs one more instruction. Failure returns in one instruction with $(T, W, D, H) = (0, w, y, 0)$. In particular neither branch adds a zero to the stack, and the successful branch no longer retains y in D.

The old MP-prelude existence-of-a-finite-run proof and this elementary loop induction therefore give a finite positive run for every incoming store and offset field, with exactly the two effects used by the new command compiler.

6.4 From these contracts to the literal binary table

The 4928 source-site outcomes in the separate symbolic tape certificate cover the 3788 leaves, with both possible outcomes of every decrement. Those contracts quantify over every unary register file and every permitted gap at least 48. Their register-engine summaries are reused only after checking every engine row is unchanged. The literal startup certificate reaches the exact source-25 frame, with candidate register $R_0 = 2$ and all other logical registers zero, after 57922 binary steps. This frame equality includes the whole finite-support tape, not just a prefix.

The arithmetic arguments above, the command compiler, and the unchanged exhaustive scheduler now prove source halting iff a numeric-calculus refutation. The positive source-site tape contracts transfer a finite source halt to a finite binary halt. Conversely, suppose a binary halt occurs within n steps after startup but the source has not halted in its first $n + 1$ instructions. Iterating the positive instruction contracts gives a binary continuation of at least $n + 1$ steps. Determinism contradicts the alleged earlier halt. This argument avoids an unproved assumption that the physical execution remains forever inside an intended frame.

All loops just used have primitive recursive trace constructors, and their correctness uses ordinary arithmetic induction. Fixed startup and binding certificates are finite arithmetic facts. The existing PA-to-calculus translation and the written syntactic calculus-to-PA interpretation can therefore be composed with this proof inside PA. The conclusion for the combined binary table is the written calibration theorem

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}^{\mathrm{combined}}) \longleftrightarrow \mathrm{Con}(\mathrm{PA}).$$

The proof has an explicit finite binding and universal arithmetic argument. It is separate from a Lean kernel-checked adaptation of all upstream arithmetic endpoint files, which has not been carried out here.

7 A complete command compiler for the combined Shift and MP changes

This section supplies the source-level correctness argument for combining two changes: Shift tests only the cached head, and successful modus ponens leaves its result in that cache. The underlying axiom calculus and all AX formulas are exactly those of the pinned PA196 source. The argument proves equivalence of the proof-search problems; a physical state count and tape simulation are separate obligations. In particular, none of the command-word counts below is a Turing-machine state count.

7.1 The changed arithmetic commands

Write a store as $\langle t, w, d, h\rangle$. Let $\langle a, w\rangle_C$ denote Cantor stack cons, and let hd, tl be its total inverse projections; both projections of zero are zero. Retain

$$I(a,b) = 4p(a,b), \qquad U(a,B) = 4p(B,a) + 2$$

and the other PA196 formula constructors. A command word is read from left to right. The modified Shift, written S, is

$$S\langle t,w,d,h\rangle = \begin{cases} \mathsf{halt}, & h = 1, \\ \langle h, \mathrm{tl}(w), d, \mathrm{hd}(w)\rangle, & h \neq 1. \end{cases}$$

For $y = d + x$, the modified modus ponens command is

$$\mathrm{MP}_x\langle t,w,d,h\rangle = \begin{cases} \langle 0, w, 0, y\rangle, & I(t,y) = h, \\ \langle 0, w, y, 0\rangle, & I(t,y) \neq h. \end{cases}$$

Both branches have a defined, terminating arithmetic effect. Only Shift can halt. AX is unchanged. Put $D = d + x$, $H = h + y$, $A = c$, and let its eight generated stack entries, listed from the top downward, be

$$\begin{aligned} q_1 &= \mathrm{head}_6(D, A, H), & q_2 &= \mathrm{head}_5(D, H), \\ q_3 &= \mathrm{head}_4(D, A, H), & q_4 &= \mathrm{head}_3(D, A, H), \\ q_5 &= \mathrm{head}_2(D, A, H), & q_6 &= \mathrm{head}_1(D, H), \\ q_7 &= \rho_8(D, A, H, d), & q_8 &= U(d, h). \end{aligned}$$

Then

$$\mathrm{AX}_{c,x,y}\langle t,w,d,h\rangle = \langle \mathrm{gate}(H, D, c), [q_1, \ldots, q_8, t] ::_C w, 0, 0\rangle.$$

Here a list before $::_C w$ means repeated Cantor cons onto w. The first seven generated entries are divisible by four. The eighth is congruent to two modulo four. The guarded gate is divisible by four as well, including when it is zero. Thus none of these generated values is one.

7.2 Stack words and the changed pop contract

The following two identities hold for every incoming store, with no restriction on its saved top t:

$$\mathrm{AX}_{c,x,y}; S^8 : \langle t,w,d,h\rangle \longmapsto \langle \rho_8(d+x, c, h+y, d), \langle t, w\rangle_C, 0, U(d,h)\rangle, \tag{1}$$

$$\mathrm{AX}_{c,x,y}; S^9 : \langle t,w,d,h\rangle \longmapsto \langle U(d,h), w, 0, t\rangle. \tag{2}$$

Indeed the first Shift tests the newly cleared zero cache; each later test up to and including the ninth tests one of $q_1, \dots, q_8$, never the saved top. This is why the second identity remains total even for $t = 1$.

Define

$$\begin{aligned} \mathsf{Push} &= \mathrm{AX}_{0,0,0}; S^8; \mathrm{MP}_0, \\ \mathsf{Pop} &= \mathrm{AX}_{0,0,0}; S^{10}, \\ \mathsf{Discard} &= \mathrm{MP}_1; \mathrm{AX}_{1,0,0}; S^9; \mathrm{MP}_0 . \end{aligned}$$

Their precise contracts are

$$\mathsf{Push}\langle t, w, d, h\rangle = \langle 0, \langle t, w\rangle_C, 0, 0\rangle, \tag{3}$$

$$\mathsf{Pop}\langle t, \langle a, w\rangle_C, d, h\rangle = \begin{cases} \mathsf{halt}, & t = 1, \\ \langle t, w, 0, a\rangle, & t \neq 1, \end{cases} \tag{4}$$

$$\mathsf{Discard}\langle t, w, 0, 0\rangle = \langle 0, w, 0, 0\rangle. \tag{5}$$

For Push, after (1) the equality tested by MP_0 compares an implication code, congruent to zero, with $U(d, h)$, congruent to two. It therefore takes the failure branch and has exactly the stated effect. For Pop, the tenth Shift is precisely the test of the saved top. For Discard, the first MP fails since $I(t, 1) > 0$, giving $\langle 0, w, 1, 0\rangle$. Identity (2) then gives $\langle U(1, 0), w, 0, 0\rangle$, and the last MP fails since $I(U(1, 0), 0) > 0$.

The old claim that Pop is total for arbitrary t must therefore be replaced by (4). An early halt when $t = 1$ is harmless in the refutation compiler, but this change cannot be silently omitted.

For $u \in \mathbb{N}$, let Offset_0 be the empty word and let $\mathsf{Offset}_u = \mathrm{MP}_u$ when $u > 0$. Then

$$\mathsf{Offset}_u\langle 0, w, 0, 0\rangle = \langle 0, w, u, 0\rangle.$$

This follows from $I(0, u) > 0$ for $u > 0$. The old generalization word remains valid without alteration:

$$\mathsf{Gen}_i = \mathsf{Push}; \mathsf{Offset}_i; S; \mathrm{AX}_{i,0,0}; S^9, \qquad \mathsf{Gen}_i\langle a, w, 0, 0\rangle = \langle U(i, a), w, 0, 0\rangle.$$

The one Shift before AX reads a zero cache. It retrieves a into the cache; AX then constructs its generalization. The proof also covers $a = 1$.

7.3 All axiom selectors remain total

For $1 \leq j \leq 6$, define

$$\mathsf{Select}_{j,D,H,A} = \mathsf{Discard}; \mathrm{AX}_{A,D,H}; S^{j+1}; \mathsf{Pop}^{8-j}.$$

From $\langle t, w, 0, 0\rangle$ this produces $\langle q_j, w, 0, 0\rangle$, with the q_j evaluated at the intended parameters. After S^{j+1} the selected q_j is the top, and the remaining stack is $[q_{j+2}, \dots, q_8, 0] ::_C w$ (with the evident empty range at the end). The following Pops preserve q_j, remove that list, and leave the final zero in the cache. Every such Pop is nonhalting because $q_j \neq 1$. The intermediate cache, which may contain another formula, is cleared by the next AX before any Shift reads it.

For $u \leq a$, the unchanged restricted-propositional selector is

$$\mathsf{Discard}; \mathsf{Offset}_u; \mathrm{AX}_{b,a-u,c}; S^8; \mathsf{Pop}.$$

It returns $\langle \rho_8(a, b, c, u), w, 0, 0\rangle$. The final Pop is safe because its preserved top is an implication code. The unchanged gate selector

$$\mathsf{Discard}; \mathrm{AX}_{c,b,a}; \mathsf{Pop}^9$$

returns $\langle \mathrm{gate}(a, b, c), w, 0, 0\rangle$. Its preserved top is again different from one. Under the original guard inequalities this is the intended bundle axiom; under a failed guard it is the harmless code zero. Thus every primitive axiom of the exact PA196 calculus still has a total selector that preserves an arbitrary underlying stack.

7.4 The new modus ponens suffix

Successful MP now puts the conclusion in h, so the old cleanup suffix is inapplicable. A dummy zero stack entry gives a shorter correct suffix:

$$\mathsf{MPSuffix}_b = S; S; \mathrm{MP}_b; S.$$

Its complete contract is

$$\mathsf{MPSuffix}_b\langle 0, [a, I(a,b), 0] ::_C w, 0, 0\rangle = \begin{cases} \mathsf{halt}, & a = 1 \text{ or } b = 1, \\ \langle b, w, 0, 0\rangle, & a \neq 1 \text{ and } b \neq 1. \end{cases} \tag{6}$$

After the first Shift, the cache is a. If $a \neq 1$, the second Shift gives $\langle a, \langle 0, w\rangle_C, 0, I(a,b)\rangle$. MP succeeds, giving $\langle 0, \langle 0, w\rangle_C, 0, b\rangle$. The final Shift either halts at $b = 1$ or returns b as the top while consuming exactly the dummy zero. No part of the arbitrary underlying stack is consumed. The dummy entry is essential: omitting it consumes the first entry of w and fails the compiler's stack-preservation invariant.

7.5 A primitive recursive compiler and universal halting word

Let $\mathcal{C}$ be the unchanged numeric calculus. A proof is a finite tree whose nodes are primitive axioms, modus ponens, or theorem generalization. Define W_p recursively. At an axiom, use the appropriate selector above. At generalization $p = \forall_i p_0$, put

$$W_p = W_{p_0}; \mathsf{Gen}_i.$$

At modus ponens, let p_0 prove a and let p_1 prove $I(a,b)$, and put

$$W_p = \mathsf{Discard}; \mathsf{Push}; W_{p_1}; \mathsf{Push}; W_{p_0}; \mathsf{Push}; \mathsf{MPSuffix}_b.$$

The first two words introduce the dummy zero. Before the suffix, the three protected entries are exactly a, $I(a,b)$, and zero.

Lemma 7.1 (Compilation invariant)**.** *If p is a $\mathcal{C}$ derivation of a, then for every t, w, the word W_p from $\langle t, w, 0, 0\rangle$ either halts or returns $\langle a, w, 0, 0\rangle$.*

Proof. Induct on the proof tree. The axiom and generalization cases use the total contracts already proved. In the MP case, Discard and Push give $\langle 0, \langle 0, w\rangle_C, 0, 0\rangle$. Apply the major-premise induction hypothesis at that stack and Push its result. Apply the minor-premise induction hypothesis at the enlarged stack and Push again. An early halt persists under appending commands. Otherwise the suffix precondition is exact, and (6) proves the claim. □

If p proves code one, define

$$\mathsf{HaltWord}_p = \mathrm{AX}_{0,0,0}; W_p; \mathsf{Pop}.$$

The first AX clears both caches of any incoming store, although it need not erase the top or stack. The compiler is independent of those values. If it returns normally, its top is one, and Pop halts by its tenth Shift before inspecting the old underlying stack. Thus $\mathsf{HaltWord}_p$ halts from *every* incoming store. This is the form required by the existing word enumerator: no assumption that its previous trials restore the zero store is needed.

7.6 Soundness and PA-provable calibration

Use the following syntactic store invariant: t, h, and every finitely decoded stack entry have $\mathcal{C}$ derivations. It places no condition on the offset register d. Code zero has a fixed derivation: the instance $\rho_8(0,0,0,0)$ is numerically zero because $I(0,0) = 0$. Thus a zero stack tail can be padded by zero entries indefinitely.

AX preserves the invariant. Its saved top is already derivable, its six heads and guarded gate are primitive axioms or zero, its restricted propositional head satisfies $d \le d + x$, and $U(d,h)$ follows by theorem generalization from the old cache h. A failed MP returns zero. A successful MP obtains y from the derivable formulas t and $I(t,y) = h$ and stores that derived formula in h. A nonhalting Shift merely moves proven formulas between the top, cache, and stack. A halting Shift exposes a derivation of one in h. Therefore a finite run from the clean store can halt only if $\mathcal{C}$ has a refutation.

The converse follows from the universal halting word and the unchanged exhaustive scheduler. The axiom-selector words, proof-tree compiler, command encodings, and extraction of a refutation from a finite run are primitive recursive. Their correctness proofs use arithmetic induction on finite lists, proof trees, and finite traces. Hence PA proves, at the command-program level,

$$\mathrm{Halt}(R_{\mathrm{combined}}) \quad \longleftrightarrow \quad \exists p \ \mathrm{Prf}_{\mathcal{C}}(p,1).$$

Composing with the existing PA-to-$\mathcal{C}$ refutation translation and the written syntactic $\mathcal{C}$-to-PA interpretation gives

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(R_{\mathrm{combined}}) \longleftrightarrow \mathrm{Con(PA)}.$$

For a particular binary table, the startup, source binding, and positive instruction simulations must additionally identify its blank-tape halting with the displayed source-program halting inside PA. A collection of finite run samples alone would not establish that identification. This section is a written proof and executable command-level check, not a claim of a new Lean kernel theorem for the modified table.

Executable supplement. The accompanying `check_commands.py` checks 64 symbolic contract cases. It expands all eight axiom-head expressions, treats the unchanged guarded gate by its exact parameter list and residue property, and uses opaque arbitrary stack tails. It distinguishes saved top one from an arbitrary non-one value, tests all nine zero/one/other MP cases, and rejects an equality if its truth is not forced by constructor equality, zero facts, or residues. It therefore does not choose an unknown branch by fiat. The general proof above supplies the universal statement; the script is an independent finite check of its command-word algebra.

8 Direct negation: a shorter source with a 197-state realization

A further arithmetic specialization replaces every occurrence of the AX macro sequence P i C. In the inherited notation, these macros push the current top, increment the new top, then combine the two values into an implication code. Their composite constructs $N(F) = I(F,1) = 4p(F,1)$ while preserving the underlying stack. There are eight such occurrences in the 201-item AX word.

For arbitrary natural t, x, y, the existing `node46` pairing core has the uniform arithmetic contract

$$(T,X,Y) = (t,x,y) \longmapsto (0,0,y + p(x,t)),$$

with W and the other framed registers unchanged. To verify it, first transfer T into X and Y. This adds t to Y and leaves $X = x + t$. The subsequent passes add the successive values $x + t, x + t - 1, \ldots, 1$ to Y, decreasing that loop rank after each pass. Thus the total addition is $t + (x + t)(x + t + 1)/2 = p(x,t)$. This is an induction on natural loop ranks, not a finite sample

test. The existing `node63` then transfers Y into the initially zero T four units at a time, giving $(T, Y) = (4Y, 0)$. Their composition is `node64`.

Consequently the closed macro

$$N_{\text{direct}} = (\text{inc}\, Y, (\text{inc}\, Y, \texttt{node64}))$$

sends $(T, X, Y) = (F, 0, 0)$ to $(4p(F, 1), 0, 0)$, since

$$p(0, F) + 2 = p(F, 1).$$

It preserves arbitrary W. The usual zero-scratch invariant holds at AX macro boundaries. Replacing all eight occurrences therefore leaves every AX output code and its stack order unchanged. All of Combined196's logical command proofs transfer directly to this variant.

The exact word/source binding is checked by `verification/check_negation.py`. The AX word shrinks from 201 to 185 macro items and the full source from 3,788 to 3,556 instructions. Compiling the selected grammar gives 202 states before five simultaneous compatible folds, and 197 afterwards. The table `machines/direct_negation197.tm` passes all 4,616 uniform source outcomes and reaches the exact clean seed after 57,922 literal steps. Its labels are sparse; its terminal label remains 196 and is excluded from the count of 197 working labels. Its SHA-256 is

`832e067d50599a0ed9ead8e19d621e6491d04faebd821e7018d3aa6708786fa6`.

Two other equal-value macro arrangements were examined: transfer F into X, set $T = 1$, and use `node64`; or use `node64` first and then add eight to T with existing increment blocks. The latter has worse address geometry in the tested layouts. The accepted 197-state table is a new complete arithmetic specialization, but it costs one more physical state than PA196 and two more than the requested maximum.

9 Why the repaired 157-node source still costs 207 states

Earlier incoming research supplies a 157-node source tree with the same 3,812-instruction ordinal control graph. Its original physical attempt failed at a transient zero-run case. This investigation reassociated the source to shorten that problematic increment suffix, then repaired a continue-to-depth-3 leaf whose path had changed from `101011` to `1010011`. A subsequent exact-CFG rotation restored a workable jump context without losing the 157-node count.

The selected source is `research/source157/source157_tailrepair_5.json`. Its resolved instruction graph is exactly equal to the original PA196 graph, including the register named by every primitive instruction and both outcomes of every decrement. Its maximum address depth is 36 and its maximum unmodified zero run is 6. Its emitted physical table passes all 4,959 symbolic instruction contracts and reaches the exact clean seed after 58,140 steps.

The physical table has 207 working states. A trace-based census identifies 153 reader controls representing the 157 source nodes, plus 54 nonreader controls. The original table has 147 readers for 159 source nodes plus 49 nonreaders. Existing helper paths keep several old source fragments alive; the generic compiler also loses contextual identifications. Exact action bisimulation leaves all 207 controls distinct. The tested finite-context direct detours and compatible two-cell folds provide no further saving for this selected allocation. These are scoped search results, not a lower bound on every possible allocation of this source.

This example is a useful failure diagnosis: a correct, smaller DAG and a correct, larger TM can coexist. A claim of a 195-state machine based only on $157 < 159$ would leave out the physical decoder, route, and reset costs.

10 An even-tag recoding that permits parity acceptance

This is a source-calculus construction and correctness theorem. It does not assert that its binary tape implementation uses fewer than 196 states. The purpose of the recoding is to make a parity test a sound acceptance test, while retaining the PA-refutation compiler.

10.1 The new formula codes

Choose an even integer $B \geq 6$ and two distinct nonzero even residues $u, e < B$. The directly relevant choices are $B = 6$ or $B = 8$, with $\{u, e\} = \{2, 4\}$. Define

$$I_B(a,b) = Bp(a,b), \qquad M_B(a,b) = Bp(a,b) + 1,$$
$$U_B(a,A) = Bp(A,a) + u, \qquad E_B(a,b) = Bp(a,b) + e.$$

Code zero is truth, and code one remains $M_B(0,0)$, the contradiction. Use exactly the PA196 heads, gate, modus ponens, and generalization, with these constructors. Negation is still $I_B(A,1)$; there is no new logical negation constructor or normalization rule.

The arithmetic interpretation decodes nonzero residues $0, 1, u, e$ as implication, bit membership, universal quantification, and equality, respectively. A code with any unused odd residue denotes falsity; a code with an unused even residue denotes truth. These cases are part of the decoder definition, not machine instructions or additional axioms. In the base-six construction every even residue is used. In the base-eight construction residue six may denote truth. The sole degenerate implication $I_B(0,0) = 0$ is handled by the fixed equivalence between truth and truth-implies-truth.

Every proper constructor component is smaller than its nonzero compound code. Thus decoding is primitive recursive by course-of-values recursion, and every variable label in the decoded formula of code c is below c. The standard gate-freshness argument remains valid, including the dual use of its parameter as a formula code and a variable label.

10.2 Why every odd acceptance yields a PA refutation

Interpret membership by arithmetic binary-bit membership and primitive equality by ordinary equality. All PA196 axiom-soundness proof templates continue to apply: only the numerical constructors have changed. For a junk-code formula the finite-comprehension template simply receives truth or falsity as its matrix. It remains a template producing a proof of an individual instance, rather than a uniform arithmetic truth predicate.

Lemma 10.1 (Odd formulas fail at the zero assignment)**.** *For every odd code a, PA proves the negation of the arithmetic translation of a after all its free variables have been replaced by zero. A proof of this negation can be constructed primitive recursively from a.*

Proof. As B is even, an odd code has an odd residue modulo B. Residue one is a membership atom $M_B(r,s)$, whose zero-assignment translation is $\mathrm{Bit}(0,0)$, refutable in PA. Every other odd residue was defined to translate to falsity. The required proofs are therefore fixed templates, preceded by decoding the residue and, in the membership case, the two indices. □

Suppose p is a derivation of an odd code in the recoded calculus. The syntactic interpretation produces a PA proof of its arithmetic translation. Substitute zero for its free variables in that finite proof and append the preceding negation proof. The result is a PA refutation. Correctness of this proof transformation is provable in PA by induction on coded derivations. Consequently

$$\mathrm{PA} \vdash (\exists p\, \exists a\, [\mathrm{Prf}_{\mathcal{C}_B}(p,a) \wedge a \text{ odd}]) \longrightarrow \neg\, \mathrm{Con}(\mathrm{PA}).$$

This argument does not claim that the recoded calculus itself derives falsity from each odd theorem. A PA refutation is sufficient for the target equivalence.

In a sound-store invariant, both live formulas t, h have derivations. Thus either of the acceptance conditions

$$h \text{ odd}, \qquad t + h \text{ odd}$$

is sound relative to PA consistency. For the second condition, an odd sum has an odd summand. Under the unchanged PA196 encoding this argument would fail: primitive equality has the odd residue three. The change to an even equality tag is essential.

10.3 Transporting the old PA-refutation compiler

Let $\sigma(0) = 0$, $\sigma(1) = 1$, $\sigma(2) = u$, and $\sigma(3) = e$. Define a recoding $\tau : \mathbb{N} \to \mathbb{N}$ by $\tau(0) = 0$ and, for $n > 0$ written uniquely as $n = 4p(a, b) + r$, by

$$\tau(n) = Bp(\tau(a), \tau(b)) + \sigma(r).$$

This is an injective primitive recursive map fixing zero and one. Injectivity follows by recovering the distinct tag and the two Cantor components, then inducting on the original codes. Both formula codes and variable labels must be transported by this same τ. In particular,

$$\tau(I(a, b)) = I_B(\tau(a), \tau(b)), \qquad \tau(U(a, A)) = U_B(\tau(a), \tau(A)),$$

and the analogous identities hold for membership and equality. The gate's formula argument c and its bound-variable label both become $\tau(c)$; injectivity preserves the two inequality guards.

There is no assumption that τ preserves the numerical order on codes. For the restricted propositional schema, use the existing finite derivation of the unrestricted schema. Its abstract construction requires modus ponens, $a \le a$, and the bounds $a, b \le I_B(a, b)$; these bounds hold for the new constructors. Thus an old restricted axiom translates to a derivable unrestricted instance even when its numerical side condition is not preserved by τ. Every other primitive axiom, MP step, and generalization step transports directly.

Replacing each axiom node by the indicated finite template defines a primitive recursive proof translation, with PA-provable correctness. Composing it with the existing PA-to-PA196 refutation compiler gives

$$\mathrm{PA} \vdash \neg\,\mathrm{Con(PA)} \longrightarrow \exists p\ \mathrm{Prf}_{\mathcal{C}_B}(p, 1).$$

10.4 Command compilation with the cached-head parity test

Retain the combined command semantics of Section 7, but halt Shift when h is odd instead of when $h = 1$, and use the new formula constructors everywhere. The proof of that section changes in three explicit places.

First, all eight AX-generated entries and the gate are even. The first seven heads are implications and the last is a universal code with the even tag u. Hence the eight- and nine-Shift identities and all selectors remain total. Push still uses a forced MP failure: implication has residue zero modulo B, whereas the cached universal formula has residue $u \neq 0$.

Second, Pop now halts exactly when its saved top is odd; otherwise it has the same stack-preserving effect. Axiom selectors use Pop only with an even preserved top, so their totality proof is unchanged.

Third, the four-command MP suffix with dummy zero halts when its antecedent or conclusion is odd and otherwise returns the conclusion with the old stack and cleared caches. The same induction on proof trees consequently gives the alternative: an early halt, or the derived formula in the top with the old stack. In particular a derivation of code one gives a universal halting word, by the final Pop if the compilation has not halted already.

The syntactic store invariant and the preceding odd-proof interpretation then yield, for the exhaustive command program,

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(R_{B,\mathrm{parity}}) \longleftrightarrow \mathrm{Con(PA)}.$$

Obtaining a small binary table remains a separate engineering and proof task: the pairing multipliers in AX and MP, all constructor residues, the acceptance routine, the source tree, the startup, and the instruction contracts must be changed consistently. A favorable source-node estimate does not certify a physical state reduction.

10.5 A residue-one acceptance test without recoding

There is a related option that leaves all PA196 formula constructors unchanged. Instead of accepting exactly cached code one, accept precisely the cached codes congruent to one modulo four. These are exactly the membership atoms $M(a, b) = 4p(a, b) + 1$.

If the unchanged numeric calculus derives $M(a, b)$, its syntactic arithmetic interpretation gives a PA proof of $\mathrm{Bit}(x_a, x_b)$. Substitute zero for the free variables and append the fixed proof of $\neg\,\mathrm{Bit}(0, 0)$. This gives a PA refutation by a primitive recursive proof transformation whose correctness is provable in PA. Conversely, a PA refutation already compiles to a derivation of the code $1 = M(0, 0)$, which satisfies the new acceptance condition. Thus accepting any proved membership atom has exactly the required consistency calibration.

The combined cached-head command compiler in Section 7 applies with the following changes: Pop halts when its saved top is congruent to one; the MP suffix halts when its antecedent or conclusion is congruent to one; all other normal-return contracts are unchanged. AX-generated heads and the gate are congruent to zero, apart from the generalization entry, congruent to two. Consequently all axiom selectors, Push, and the generalization word remain total. The compilation theorem still permits an early halt, and a proof of code one still provides a universal halting word. Soundness extracts the derivation of the accepted membership atom and uses the zero-substitution translation just described.

The criterion must be imposed on the cached head alone. Testing $t + h \equiv 1 \pmod 4$ would also accept residues $(2, 3)$ or $(3, 2)$, involving a universal formula and an equality formula. The membership-atom argument does not justify those extra cases. A concrete implementation needs a certified modulo-four guard and its complete source/tape contracts; no state saving is claimed without them.

11 Physical experiments for the parity architecture

The mathematical recoding in Section 10 is a proposed architecture, not the encoding currently present in the parity-control table. The control experiment changes the Shift prelude to leave $T = h$, $H = 0$, and $X = t + h$; removes the final increment of X; and uses the source guard

$$(\mathrm{dec}\, X, ((\mathrm{dec}\, X, \mathrm{continueTo}\, 2), \mathrm{halt})).$$

It consumes X in pairs and halts on odd inputs. The source contains 3,801 sites and 4,944 outcome contracts.

Return detector	States	Failing outcomes	Meaning
Original seven-zero detector	197	38	19 source sites fail
Shortened six-zero detector	196	105	Invalid runtime certificate
Extended eight-zero detector	198	0	All 4944 local contracts pass

The original runtime has intermediate returns through the root decoder on 19 source sites. They are harmless with its original root but not with the new parity root. The eight-zero detector repairs these local contracts at the cost of one state. It does not discharge startup: the old exact seed was not reached in the explicit two-million-step test. Earlier clean frames were observed with different register contents, so this is an unresolved bootstrap obligation, not a proof that the table can never work.

Moreover, these parity tables still use the old four-residue AX/MP constructors. They have not integrated the even-tag encoding needed by the logical argument. The table is deliberately placed under `research/parity_and_engine`, not under the complete machines.

The base-6 and base-8 grammar searches charged for the new multiplier macro and equality tag. Their best tested layout estimates did not improve on the original physical count; some were around 200 even before a fully validated parity guard and bootstrap. These estimates are neither certified machine counts nor optimality results. The parity theorem remains useful because it identifies a valid simpler acceptance condition and precisely states the encoding change needed to support it.

12 ConciseIPRA: useful ideas and the strength barrier

The current ConciseIPRA README [4] describes substantially more than a small checker file. It includes raw one-term transport, direct progress rules, a seven-rule single-introduction presentation, clause-based five- and four-family forms, and three-family forms such as Use/Intro/Name and Use/Intro/Open on full normal forms. It distinguishes Lean proofs of calculus equivalence from Python measurements of checker byte length.

These results provide useful methods: derive redundant rules, use a normal form that exposes common code, and separate logical equivalence from the cost of its implementation. They do not automatically produce a binary Con(PA) machine with the same number of rules, bytes, or source nodes. A normalizer, substitution machinery, proof enumerator, and input encoding all have a physical state cost.

There is also a logical distinction. A machine searching for an iPRA refutation naturally calibrates to iPRA consistency. Its equivalence to PA consistency requires an additional proof-theoretic argument; one cannot replace the target theory by name. The prior incoming packet `pa196_research_2026-11-13.zip` already records the HA-strength generalization obtained by allowing compound induction progress formulas, with the corresponding PA consistency calibration. That is relevant prior work. No smaller binary implementation of that strengthened checker was completed here.

13 Rejected shortcuts and exact search limits

1. **Combining the 208-state merges.** The original packets remove the same native 172/211 distinction. Renaming the surviving state does not make a second independent reduction.

2. **Deleting a reset state.** The nominal 195-state six-zero version fails the first source instruction's uniform runtime contract. The clean source address has six zeros, and a transient runtime marker creates the misleading interior boundary. This refutes the proposed simulation certificate; it does not decide the blank-run behavior of every 195-state machine.

3. **Accepting all odd codes in the original encoding.** Equality has residue 3. The attempted clone interpretation fails because equality can distinguish objects that the proposed membership relation cannot separate. Even-tag recoding is needed for the parity argument.

4. **Keeping an old compiler after changing commands.** The new Pop can halt on a saved top of 1, and the MP suffix consumes an extra stack entry unless it is supplied a dummy zero. Both facts are explicitly handled by the compiler in this report.

5. **Removing native state 198.** All 780 tested one-row redirects of its sole incoming edge failed the 72 selected register-file cases, even when the allowed spare tail was relaxed. This rules out that finite family, not every possible redesign of the carry engine.

6. **Five-control synthesis of a six-control carry block.** Unrestricted bounded synthesis timed out. A restricted version preserving the original nonzero scan rows was unsatisfiable on the selected sample contracts and bounds. No engine minimality conclusion follows.

7. **Fresh-label and weak-head proposals.** Enlarged fresh labels have an unproved low-label vacuity/refutation-translation obligation. They have not been disproved merely by that gap. Countermodels to restricted implicational fragments do not establish incompleteness of the entire quantifier-and-gate calculus.

New finite-window helper searches were also run on PA196, the combined table, an earlier structurally different 196-state bracketing, and its combined variant. The helper may perform arbitrary allowed bit writes and left/right moves within the stated finite window before rejoining the old transition endpoint. No compatible net-saving shared helper or direct detour merge was found in those searches. This is a restricted absence of candidates; it is not evidence that 196 is minimal.

A separate exact-CFG physical-rotation search, keeping the AX region fixed, exhausted its connected component of 1,000 layouts at depth 16. All had 196 states after the tested duplicate-row and predecessor-bit folds. This finite component is defined by the allowed rotation moves, not by all possible source bracketings or all equivalent Turing machines.

For Combined196, an independent unary/list symbolic checker also rederived its engine summaries and passed all 4,928 cases, recording 1,204,510 symbolic events. Intersecting its source contexts with an independent blank-window closure left 2,214 control/window pairs. An engine-inclusive screen then considered one deleted state and up to two arbitrary changed rows, requiring the original transition endpoint within 16 child steps and within the five cells at offsets $-2, \ldots, 2$. It tested 38,220 ordered state deletions, 118,490,624 first-row assignments, and 576,694,720 second-row assignments. All 635 representative passes failed on the full admitted context set. The two narrower deletion templates also had no survivors. The exact context hash, programs, scope, and counts are under `research/engine_screen`. This is a negative result for the stated local simulation templates, not for every possible 195-state machine.

14 Reproduction and certificate boundaries

From the packet directory run:

```
python3 verify.py
python3 verify.py --extended
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The optional `python3 build_report.py` wrapper performs the two LaTeX passes and can build a local format when a TeX installation supplies the packages but has no prebuilt format. The first command checks the pinned PA196 and Combined196 tables, exact startups, all their symbolic source outcomes, the canonical comparison, the 64 symbolic command-algebra checks, and the finite source-macro binding. The second also checks the source-CFG identity and 207-state realization of the repaired 157-node tree, together with the 197-state direct-negation variant. Python needs only its standard library. The command and binding supplements are also executed under optimization to ensure their essential conditions are not lost with Python assertions.

The scripts output machine-readable receipts under `receipts`. `SHA256SUMS.json` records the packaged file hashes. The upstream Lean source is included selectively as a reference and proof dependency; this packet's Python verification is not a substitute for rebuilding the upstream Lean project. The written changed-command proof is new work, while its AX, scheduler, pairing, register-engine, and proof-translation dependencies are explicitly inherited from the pinned sources and prior written arguments.

The deliverable records a failed strict state-reduction goal and useful verified intermediate results. It supplies no certified transition table with fewer than 196 working states. A future success must provide that table, its actual physical census, an exact blank startup, a uniform runtime proof, and the PA-internal proof transformations together.

Attribution for the reproduced proof appendix

The following section is reproduced unchanged from `pa_tm_research_2026-11-13.zip`, member `pa_tm_research_2026-11-13/report/pa_reverse_translation.tex`. Its exact archive and member hashes are in `reference/prior_proofs/provenance.json`. It is prior work supplied among the requested inputs. Its implementation obligations and conditional transfer statement are retained as written. Sections 6 and 7 of the present report supply the changed source and command arguments for Combined196; the new machine still has no newly exported Lean endpoint. The appendix's encoded negations use the fixed PA theorem $\neg B(x_0, x_0)$, so their translation has the intended falsity behavior rather than an arbitrary propositional constant.

A The reverse consistency implication

The published PA196 endpoint explicitly supplies

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

Its external soundness and nonhalting theorems do not, merely by being formalized in Lean, assert the opposite implication inside PA. We give a syntactic interpretation of the command calculus and a conditional transfer theorem for the physical compilation. The calculus interpretation below is an ordinary mathematical proof. Its application to the exact arithmetized tape predicate still has the explicit implementation and binding obligations listed at the end of this section. No new closed Lean theorem of the internal biconditional is claimed.

A.1 An arithmetical interpretation of the exact calculus

Write $B(u, v)$ for the arithmetical predicate asserting that the vth binary digit of u is one. Thus the first argument is the container, as in the convention for M in PA196. For convenience work first in a fixed definitional extension PA^+ for the primitive recursive functions needed to express binary digits and exponentiation. Fix their PA proofs of totality and the usual primitive recursive elimination of this finite definitional extension from proofs. In particular,

$$\mathrm{PA}^+ \vdash B(u, v) \longrightarrow v < u, \qquad \mathrm{PA}^+ \vdash \neg B(u, u). \tag{7}$$

The first implication follows from $2^v \le u$ and $v < 2^v$.

The translation T sends truth to $0 = 0$ and commutes with implication and universal quantification. On atoms it is

$$T(\mathsf{M}(a, b)) = B(x_a, x_b), \qquad T(\mathsf{E}(a, b)) = (x_a = x_b), \qquad T(\mathsf{U}(a, F)) = \forall x_a\, T(F).$$

The numerical decoder reads $\mathsf{U}(a, F) = 4p(F, a) + 2$ with its transposed arguments. All its recursion is bounded by the input code. The one degenerate implication constructor is $\mathsf{I}(0, 0) = 0$; here $T(0) = 0 = 0$ is provably equivalent to $T(0) \to T(0)$, and a fixed logical proof bridges the syntactic mismatch. Other implication constructors commute literally with T.

Every variable label in a formula of code c is less than c. This is the syntactic bound proved in `PA196/Logic/Decode.lean`; in particular x_c does not occur in $T(c)$. No assertion that $T(c)$ is true is involved.

A.2 Uniform finite-comprehension proofs

Lemma A.1 (A proof-producing comprehension schema)**.** *For every arithmetical formula $\psi(z, \vec{u})$, with auxiliary variables chosen fresh,* PA^+ *proves*

$$\forall n\, \exists s < 2^n\, \forall z < n\big(B(s, z) \leftrightarrow \psi(z, \vec{u})\big). \tag{8}$$

There is a primitive recursive function of the code of ψ that produces such a proof, and PA proves the syntactic correctness of this proof-producing function.

Proof. Use the displayed existential formula as the induction formula in n. At $n = 0$ take $s = 0$. At the step, take $s + 2^n$ if $\psi(n, \vec{u})$ and take s if $\neg\psi(n, \vec{u})$. The binary-digit update laws prove the required statement at $n + 1$; the old witness satisfies $s < 2^n$, so adding 2^n introduces exactly the new digit. This is ordinary classical reasoning in PA.

The proof consists of a fixed finite logical template, the fixed binary arithmetic lemmas, and one induction-axiom instance built from ψ. Variable renaming, substitution, formation of that instance, and proof concatenation are primitive recursive syntax operations. Verifying that the constructed instance satisfies the PA induction schema is a syntactic calculation. Consequently PA verifies the output as a proof uniformly in the code of ψ; no uniform arithmetical truth predicate is used. □

Applying the lemma to $\big(B(A, z) \vee z = b\big) \wedge \psi(z, \vec{u})$, with cutoff $A + b + 1$, gives

$$\exists s\, \forall z \left[B(s, z) \leftrightarrow \big((B(A, z) \vee z = b) \wedge \psi(z, \vec{u})\big)\right]. \tag{9}$$

Equation (7) shows that every eligible z lies below the cutoff, while a witness less than 2^{A+b+1} has no digits beyond it. This is the finite pointed-separation schema needed by the gate.

A.3 Translation of the gate and the logical heads

The six PA196 heads and the restricted propositional head are valid first-order or equality schemata after translation. Finite case distinctions on coinciding variable labels give uniform proof constructors. The domination restriction on the fourth argument of the propositional head restricts the available instances, but plays no role in their validity.

For the gate, keep the notation of the PA196 paper:

$$\Delta = \mathsf{E}(a, b) \to \mathsf{U}(c, \mathsf{E}(a, b)), \quad V = \mathsf{M}(a, b) \to \mathsf{U}(c, \mathsf{M}(a, b)),$$

$$P = c \to \mathsf{M}(c, b), \quad S = \mathsf{U}\big(b, (\mathsf{M}(a, b) \to P) \wedge (\mathsf{M}(c, b) \to c)\big), \quad Q = \mathsf{U}(c, S \to \neg P).$$

The returned formula is $\neg(\Delta \to (V \to Q))$ under $c \neq a, b$, and truth otherwise.

If the guard fails, the translation is immediately provable. If it holds, x_c is absent from the translated equality and membership atoms, so $T(\Delta)$ and $T(V)$ have elementary vacuity proofs. Put $\varphi = T(c)$; x_c is also absent from φ by the code bound. For $a \neq b$, equation (9), with $\psi(z) = \varphi[z/x_b]$, constructs a witness s satisfying

$$\forall x_b \left[(B(x_a, x_b) \to (\varphi \to B(s, x_b))) \wedge (B(s, x_b) \to \varphi)\right]$$

and also $\varphi \to B(s, x_b)$ at the distinguished original value of x_b. Thus $\exists x_c(T(S) \wedge T(P))$ has a PA proof.

The collision $a = b$ deserves a separate check: the first membership atom inside S then becomes $B(x_b, x_b)$, which PA refutes. Use pointed comprehension on the singleton distinguished point; it proves the second conjunct and the point clause, while the first conjunct is vacuous. This handles the collision without incorrectly treating x_a as an unchanged parameter under the x_b binder.

Classical first-order reasoning now gives $\neg T(Q)$ and hence $T(\neg(\Delta \to (V \to Q)))$. Every step is part of a uniform finite proof template. This proves the following syntactic statement.

Lemma A.2 (Refutation translation)**.** *Let $\mathrm{Prf}_C(p, f)$ be a primitive recursive proof predicate for exactly the PA196 numeric calculus. There is a primitive recursive function F for which PA proves*

$$\mathrm{Prf}_C(p, 1) \longrightarrow \mathrm{Prf}_{\mathrm{PA}}(F(p), \ulcorner \bot \urcorner).$$

Proof. Primitive recursion over the finite derivation translates each axiom using the preceding constructors, each modus ponens by proof concatenation, and each generalization by the corresponding PA rule. Use universal closures at intermediate open conclusions. The code-zero constructor collision is bridged by the fixed equivalence already described. The result is a proof in PA^+ of the translated conclusion. At code 1 this is $B(x_0, x_0)$, whose negation has the fixed proof in (7). Append that proof and eliminate the definitional extension.

Correctness is proved by induction on derivation syntax, using only syntactic checking of the produced proofs. This is neither a reflection principle for PA nor an induction on the truth of arbitrary PA formulas. □

A.4 From a finite halting trace to a refutation

The command invariant in `PA196/Logic/CommandSoundness.lean` can be made explicitly proof-producing. Retain a derivation code for each live formula t, h and a sequence of derivation codes parallel to the numeric stack. Initially all formulas are truth. Stack underflow returns truth with its fixed proof. A successful MP appends an application of modus ponens; a failed MP supplies truth. AX supplies its six heads, restricted propositional head, generalization of the old h, saved old top, and guarded gate or truth. These are primitive recursive updates of finite data.

When Shift halts, $t + h = 1$. Since these are natural numbers, one of t, h is exactly 1, and its retained code is a derivation of the contradiction. Notice that this extraction requires no assertion that a derivation of 1 cannot exist.

For the tape machine, the finite startup and total positive-length instruction contracts permit extraction along a finite trace. Determinism identifies each completed instruction boundary with the source execution. A tape trace of length N has at most N completed source instructions, since every instruction contract has positive length. No nonhalting engine macro has an internal halting exit: its totality and designated exit are part of its contract. Thus a halting trace reaches the source Shift test, and the preceding annotated invariant returns a calculus refutation. Bounded traversal of the trace and the primitive recursive syntax updates make the extractor primitive recursive.

This gives the following useful transfer criterion. The published development contains substantial physical and internal-compilation infrastructure for its application. A modified table must satisfy the same stated hypotheses; the existence of finite test runs does not supply them.

Theorem A.3 (Internal equivalence from certified compilation)**.** *Suppose a binary table M has a PA-verifiable finite startup and total positive-length instruction contracts for the PA196 command search, including its unique halting test. Suppose also that the PA196 compiler from a coded PA refutation to a finite halting execution is retained. Then*

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

Proof. The forward implication is supplied by the refutation-to-execution compiler, contraposed. For the reverse implication, PA verifies the bounded extraction from a halting trace to a calculus refutation and then applies Lemma A.2. Hence PA proves that halting yields a PA refutation. Contraposition is the desired reverse implication. □

The theorem identifies the exact extra obligations behind an internal biconditional. External soundness, finite testing of selected runs, and a theorem of two-sided independence are each insufficient by themselves. Conversely, no uniform truth predicate and no PA reflection assumption are needed once these syntactic compilations and total physical contracts are supplied.

Remaining artifact-level obligations. The following concrete work is not supplied by this section and must not be inferred from the existence proof of the calculus compiler:

1. Fix the certificate encodings and connect Prf_C and $\mathrm{Prf}_{\mathrm{PA}}$ above to the exact checker and consistency sentence used in the published arithmetic statement. Record and verify the encoding conversions.
2. Implement the proof-producing templates and their arithmetized correctness proofs, including the code-zero implication collision, fresh variables in comprehension, and the coincident-label gate case. The mathematical template construction above does not constitute a compiled Lean implementation of these transformations.
3. Bind the reverse trace extractor to the literal machine's exact arithmetic trace predicate. Verify the startup, every relevant positive instruction contract, termination of its bounded traversal, and extraction of the Shift certificate within the chosen arithmetization.
4. For a reordered AX body, verify its physical compilation and update the axiom-selection ranks in the forward certificate compiler. For push order $(h_1, h_2, h_3, h_5, h_6, h_4)$ the ranks from the top are respectively $(6, 5, 4, 3, 2, 1)$ in that displayed order; equivalently, h_4 has rank 1, h_5 rank 3, and h_6 rank 2. The propositional head, generalization and saved top retain ranks $7, 8, 9$.

The fixed permutation preserves the exact set of available axiom instances. It changes which head a particular command word selects; the appropriate equivalence is through the adjusted proof compiler, not equality of the literal command executions.

References

[1] James Chang, *BusyBeaver: PA196*, pinned source and paper, commit `407cdc356f6b586635e71ac92be394255f4bfea4`. https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4.

[2] James Chang, *BusyBeaver: PA209*, historical source and paper, commit `69d98241a721ead20b4f0b17df9062950b40fbfe`. https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe.

[3] Vladimir Reshetnikov, *ProveIt, docs/incoming*, 18 pinned research archives, commit `dc523e7b06ffb414093c90aa035500ca283e5f75`. https://github.com/VladimirReshetnikov/ProveIt/tree/dc523e7b06ffb414093c90aa035500ca283e5f75/docs/incoming.

[4] Vladimir Reshetnikov, *ConciseIPRA*, current README and linked calculus developments at the same ProveIt commit. https://github.com/VladimirReshetnikov/ProveIt/tree/dc523e7b06ffb414093c90aa035500ca283e5f75/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[5] Earlier research packets included in [3], especially `pa196_research_2026-11-13.zip`, `pa208_research_2026-10-29.zip`, and the prior comparison/reduction reports. Exact current archive and selected-member hashes are listed in this packet's provenance directory. Prior proof arguments and code are reused with their original notices preserved.